

Module 3 - Summary Video – Transcript

Note: This transcript is auto-generated from the Teams application; it may include errors.

Hello everyone and welcome. I am Maria Konte, the instructor of the Computer Networks course. In this video, we focus on the protocols an organization uses to move traffic efficiently within its own network, which we refer to as intradomain routing. We start by taking a look at where routing fits in the Internet Protocol stack. We take a look at the tasks that the router is responsible for. We break that down into forwarding and routing, and also we contrast intradomain versus interdomain routing.

We take a look at two different flavors of intradomain routing, namely link state and distance vector, and we wrap up with practical issues such as the count-to-infinity problem and hot potato routing.

As a practical scenario, let's consider a host in an enterprise network with a browser application making a web request to a server that is hosted in a content distribution network. The transport layer offers end-to-end communication between the processes that are running at the two different hosts, but the transport layer doesn't tell us how the network traffic will cross the intermediate routers and networks to complete this web request.

More specifically, once the browser has an IP address for the destination, typically this is learned via another protocol, which is the DNS protocol. The next question is how does each packet actually travel hop by hop through routers across ISP networks and into the CDN network to reach the web server?

Therefore, the packets aren't just moving through routers; they are moving through multiple administrative domains. By the term administrative domain, we refer to a network that is operated by a single organization under a shared set of routing policies, so each domain has its own policies and its own view of the world. Therefore, routing has to work within a domain, and also it has to work between domains. Let's talk about these two different cases.

Inside the single domain, routers run an intradomain routing protocol or an internal gateway routing protocol to compute the best internal paths. But to move between domains, edge routers use interdomain routing, mainly BGP. We will talk about that.

To choose which neighboring domain to send the traffic to next. So IGP, or as we said, Internal Gateway Protocol, is responsible for moving the packets across a domain, and BGP

moves the packets from domain to domain, repeating until the packets reach a destination domain, for example the CDN network in this case, and then finally the server. So by a path, we mean the sequence of routers and links a packet traverses within a domain. If the final destination is outside the domain, the IGP computes the internal path to the border router, and then BGP determines which neighboring domain to send the packet to next.

So now that we have seen which protocols operate where, the next question to ask is what does an individual router actually do to make this work? So this takes us to the router's two tasks or two responsibilities, which are forwarding and routing. So when we refer to the router's task, we're really thinking about two different jobs happening at two different speeds: forwarding and routing. In our example, the browser sends traffic towards the web server and the packets are going through multiple routers along the way.

At each router, forwarding is the data-plane action. The router checks the destination address of the packet, consults its forwarding table, chooses the next hop and outgoing interface, and moves the packet from the input port to the output port.

So all these operations are happening per packet. So it's a process that happens locally and in short time scales. But the router needs that forwarding table in the first place. So this is where routing comes in, and that represents the control plane. Routing protocols enable the routers to exchange information to learn about the network, compute the best paths according to some metrics, and then build or update the forwarding table. Now when changes happen, for example, let's say that we have a link failure or a cost update, then routers update and converge to new paths.

Now with this distinction in place, we're ready to focus on intradomain routing, and we will take a look at two different flavors, link state and distance vector. Now in link state, the routers build a shared view of the network.

And that is the nodes, the links, the cost for each link, which can be the bandwidth, delay, or an operator-defined metric. The goal for every router in the network is to compute paths, as we mentioned, the sequence of routers and links that the packet will follow, and select the best path, meaning the one with the lowest total cost under a specific metric. Then it uses the best path to populate its forwarding table with the next hop information for every destination, which reflects the first hop on the best path.

Now, to make that more concrete, let's select one router as our source and define the problem that we're attempting to solve. So in link state, we describe the computation from the

perspective of one router at a time. Here we treat U as our source router, and it represents every other router in the network. So the question is, from the source view, what is the least-cost path to every other node in the network? If U can answer that, then it can determine for every destination which neighbor should be the next hop.

Now to compute those least-cost paths, we represent the network as a graph where routers are represented as nodes, links as edges, and every edge is annotated with a cost. So once we have this representation, link-state routing is essentially an algorithmic problem using the shared topology and the link costs. Each router computes the least-cost path from itself to all other nodes and then turns that result into the forwarding table entries. With the graph model in place, we're ready to take a look at the specific shortest-path process that link-state routing uses, which is Dijkstra's algorithm.

So first we select a source router. As we mentioned, here it is U, and our goal is to find the lowest-cost path from U to every other node in the network. Now to do that, the algorithm keeps track of three pieces of information: D, our current best-known cost to every node; P, which is the predecessor that tells us how we got that best path; and N prime, which is the set of nodes whose shortest paths are already confirmed. With these three pieces of information in place, let's see how the algorithm initializes from the source.

So initialization is straightforward. Here we start with N prime to include only node U. Then we set the distances to U's direct neighbors equal to the link cost. So in this case X is 1, for V is 2, and for W is 5, and everything else starts at infinity because we do not know the best route towards them at first. We also set the predecessor for each node to U, since it is the best-known path that is currently only one hop. And next, we go through iterations where repeatedly we select the closest node that we know so far, add it to our N prime set, and update the nearby distances.

Now as the algorithm runs, we keep adding the next closest node into N prime and we relax its neighbors. We stop when all nodes are in N prime. That means every shortest path from the source is now confirmed.

The final result is a shortest-path tree rooted at the source, and the router turns that into a forwarding table by keeping, for each destination, the first hop on the shortest path. Finally, the simple implementation has a complexity of $O(N^2)$ because we do N rounds, and for each round we scan the remaining nodes, $N - 1$, $N - 2$, and so on, to find the next minimum.

Now OSPF, or in other words Open Shortest Path First, is one of the most common examples of link-state routing algorithms that are used inside an administrative domain. The key idea is the same one as we just talked about, where routers need a consistent map of the network so they can compute best paths using Dijkstra's algorithm. In OSPF, routers share that map by flooding link-state advertisements or LSAs, which include information about their local links and the costs.

Every router in an area collects these advertisements into a link-state database, and then each router runs Dijkstra's algorithm locally to compute shortest paths and install next-hop entries into their forwarding tables. The diagram here highlights the fact that OSPF is hierarchical.

A single AS is split into areas that are connected through a backbone area and area border routers that tie those different areas together, so the advertisements don't have to flood the entire domain.

Now, on the other hand, distance-vector routing is the second approach when it comes to internal gateway protocols, and the goal that it has is the same as in link-state routing in the sense that the goal is to find least-cost routes, but in this case without building a global network map. Instead, each router keeps track of a distance vector, which represents its own view of the entire network, essentially its current best cost to reach every destination in the network.

Routers only exchange these vectors with their neighbors, so information spreads hop by hop through the network. The logic comes from the Bellman-Ford equation: my best cost to reach destination Y is the minimum over all neighbors V of the link cost to V plus V's best-known cost to Y. Or in other words, for each node we try each neighbor as the first hop, and we select the one with the cheapest cost. Now next, let's see what that looks like operationally in the sense of how routers actually update their tables over time.

So here we have the distance-vector update process, where the routers exchange their distance vectors with neighbors and they recompute using the following rule: the cost via neighbor equals the link cost to that neighbor plus the neighbor's advertised cost to the destination.

So if a router finds a cheaper value, it updates its table, and if anything has changed, it readvertises the new distances to its neighbors. So in this toy example that we show here, each node starts by knowing only its direct link costs.

After the first exchange, nodes can discover better routes by combining what neighbors report, so improvements can spread one hop at a time. Now, after a round of exchange and recompute, the network converges.

Now, distance vector converges when routers stop changing their tables, meaning that no router has any new updates to send. At that point, each router knows its least-cost distance to every destination, even though no one ever built a full topology map of the entire network.

The practical output is the forwarding decision, so for each destination it is able to choose the next hop that achieves the best cost. In this toy example, node X learns that it can reach Z with cost 3 via Y, which is less than the direct cost of seven, and that is reflected in the final tables. Now this looks great when costs are stable or when routes are getting cheaper, but the distance vector has a weakness when link costs increase or links fail. So now we will see how bad news travels slowly, leading to the so-called count-to-infinity problem.

Now the count-to-infinity problem is the failure of distance-vector routing when a link cost increases a lot or a link is failing. In this example, the cost from Y to X jumps from 4 to 60, and router Y immediately looks for another route to X.

And it may believe that it can reach X via Z because Z was previously advertising a path to X, but Z may still believe that it can reach X via Y. So now we have this loop of mistakes: Y thinks that it can go through Z, and Z thinks that it can go through Y. As they exchange these updates, the cost doesn't jump to the true value right away. It is only increasing by one at a time. So that's why it's called the count-to-infinity problem, reflecting the fact that the network can take many iterations to converge after bad news happened, like for example a link failure or a link cost increase. So how do we tackle this problem? This leads us to a technique that is called poison reverse. So poison reverse is a simple fix. The idea is: if Z's best route to X goes through Y, then Z advertises to Y that its distance to X is infinity. Even though this is not true, it's a deliberate lie to stop Y from thinking that they can reach X via Z because that would create an immediate back-and-forth loop.

The poison reverse technique blocks this pattern and helps the network converge faster when a link cost increases or fails.

Now the limitation is the fact that it can prevent this phenomenon in simple topologies, like for example when we have three nodes, but in the case where we have more complex topologies, it might not be enough.

And finally, let's talk about hot potato routing. So we just saw that the poison reverse technique helps prevent simple loops during convergence. But there is an additional practical question or problem, which is which exit should an ISP use in the case that it has multiple equally good ways out of the network.

Hot potato routing comes in and reflects the fact that when an ISP has multiple egress points, it could use them to reach the same destination, as for example in the figure that we see on the slide. Let's assume that we have a router located in Dallas that can send traffic out of the network, either through the router located at San Francisco or New York. And from a BGP perspective, let's assume that both exits may look equally good. Now, when that happens, the ISP applies a simple rule to select the closest exit.

Where closest means the exit with the lowest IGP cost inside the ISP. So if the intradomain cost to San Francisco is, for example, 9 and the cost to New York is 10, then Dallas forwards the traffic through San Francisco because that gets the traffic out of the ISP network sooner. Operators prefer this because it's practical, in the sense that it minimizes how long the traffic consumes internal resources like links or other resources. Now the catch here is that hot potato routing ties interdomain behavior to intradomain costs. If IGP costs change inside the AS, the closest exit can also change as well, and therefore that can result in shifting traffic paths. In return, this can even trigger visible BGP updates.

We will talk about all of this when seeing interdomain routing again.

In summary, to wrap this up, we talked about how the transport layer offers protocols that set up the connection between applications, but it is the IP layer that actually is responsible for carrying packets hop by hop across routers.

We also talked about the two roles that the router has, which are forwarding, moving packets quickly using the forwarding table, and routing, which is the control-plane type of work that computes and updates tables. We also saw that routing is split into interdomain and intradomain because it crosses multiple types of networks such as enterprise, ISP networks, and CDN networks. So inside the domain, we took a look at two different gateway routing protocols, namely the link-state routing protocol and the distance-vector routing protocol.

And finally, we took a look at practical issues such as the count-to-infinity problem, poison reverse, and hot potato routing.

Thank you for watching.